2 Getting Started

Let's get started and try out spaCy! In this exercise, you'll be able to try out some of the 60+ available languages.

Part 1: English

- Use `spacy.blank` to create a blank English (`"en"`) `nlp` object.
- Create a `doc` and print its text.

INPUT

```
# Import spaCy
import spacy

# Create the English nlp object
nlp = spacy.blank("en")
# Process a text
doc = nlp("This is a sentence.")

# Print the document text
print(doc.text)
```

Run code | Submit

OUTPUT

```
This is a sentence.
✔ Well done!
```

Show solution | Reset

Part 2: German

- Use `spacy.blank` to create a blank German (`"de"`) `nlp` object.
- Create a `doc` and print its text.

INPUT

```
# Import spaCy
import spacy

# Create the German nlp object
nlp = spacy.blank("de")

# Process a text (this is German for: "Kind regards!")
doc = nlp("Liebe Grüße!")

# Print the document text
print(doc.text)
```

Run code | Submit

OUTPUT

```
Liebe Grüße!
✔ Sehr gut! :)
```

Show solution | Reset

Part 3: Spanish

- Use `spacy.blank` to create a blank Spanish (`"es"`) `nlp` object.
- Create a `doc` and print its text.

INPUT

```
# Import spaCy
import spacy

# Create the Spanish nlp object
nlp = spacy.blank("es")

# Process a text (this is Spanish for: "How are you?")
doc = nlp("¿Cómo estás?")

# Print the document text
print(doc.text)
```

Run code | Submit

OUTPUT

```
¿Cómo estás?
✔ ¡Perfecto! Let's move on to documents, spans and tokens.
```

Show solution | Reset

3 Documents, spans and tokens

When you call `nlp` on a string, spaCy first tokenizes the text and creates a document object. In this exercise, you'll learn more about the `Doc`, as well as its views `Token` and `Span`.

Step 1

- Use `spacy.blank` to create the English `nlp` object.
- Process the text and instantiate a `Doc` object in the variable `doc`.
- Select the first token of the `Doc` and print its `text`.

INPUT

```
# Import spaCy and create the English nlp object
import spacy

nlp = spacy.blank("en")

# Process the text
doc = nlp("I like tree kangaroos and narwhals.")

# Select the first token
first_token = doc[0]

# Print the first token's text
print(first_token.text)
```

Run code Submit

OUTPUT

```
I
✔ Nicely done!
```

Show hints Show solution Reset

Step 2

- Use `spacy.blank` to create the English `nlp` object.
- Process the text and instantiate a `Doc` object in the variable `doc`.
- Create a slice of the `Doc` for the tokens “tree kangaroos” and “tree kangaroos and narwhals”.

INPUT

```
# Import spaCy and create the English nlp object
import spacy

nlp = spacy.blank("en")

# Process the text
doc = nlp("I like tree kangaroos and narwhals.")

# A slice of the Doc for "tree kangaroos"
tree_kangaroos = doc[2:4]
print(tree_kangaroos.text)

# A slice of the Doc for "tree kangaroos and narwhals" (without the ".")
tree_kangaroos_and_narwhals = doc[2:6]
print(tree_kangaroos_and_narwhals.text)
```

Run code Submit

OUTPUT

```
tree kangaroos
tree kangaroos and narwhals
✔ Good job!
```

Show hints Show solution Reset

4 Lexical attributes

In this example, you'll use spaCy's `Doc` and `Token` objects, and lexical attributes to find percentages in a text. You'll be looking for two subsequent tokens: a number and a percent sign.

- Use the `like_num` token attribute to check whether a token in the `doc` resembles a number.
- Get the token *following* the current token in the document. The index of the next token in the `doc` is `token.i + 1`.
- Check whether the next token's `text` attribute is a percent sign "%".

INPUT

```
import spacy

nlp = spacy.blank("en")

# Process the text
doc = nlp(
    "In 1990, more than 60% of people in East Asia were in extreme poverty. "
    "Now less than 4% are."
)

# Iterate over the tokens in the doc
for token in doc:
    # Check if the token resembles a number
    if token.like_num:
        # Get the next token in the document
        next_token = doc[token.i + 1]
        # Check if the next token's text equals "%"
        if next_token.text == "%":
            print("Percentage found:", token.text)
```

Run code Submit

OUTPUT

```
Percentage found: 60
Percentage found: 4
✔ Well done! As you can see, you can do a lot of very powerful analyses
using the tokens and their attributes.
```

Show hints Show solution Reset

Mark as completed Next

6 Pipeline packages

What's **not** included in a pipeline package that you can load into spaCy?

- ○ A config file describing how to create the pipeline.
- ○ Binary weights to make statistical predictions.
- ◉ The labelled data that the pipeline was trained on.
- ○ Strings of the pipeline's vocabulary and their hashes.

Submit

That's correct! Trained pipelines allow you to generalize based on a set of training examples. Once they're trained, they use binary weights to make predictions. That's why it's not necessary to ship them with their training data.

✓ Completed Next

7 Loading pipelines

The pipelines we're using in this course are already pre-installed. For more details on spaCy's trained pipelines and how to install them on your machine, see the documentation.

- Use `spacy.load` to load the small English pipeline `"en_core_web_sm"`.
- Process the text and print the document text.

INPUT

```
import spacy

# Load the "en_core_web_sm" pipeline
nlp = spacy.load("en_core_web_sm")

text = "It's official: Apple is the first U.S. public company to reach a $1 trillion mar

# Process the text
doc = nlp(text)

# Print the document text
print(doc.text)
```

Run code Submit

OUTPUT

```
It's official: Apple is the first U.S. public company to reach a $1 trillion mar
ket value
✔ Well done! Now that you've practiced loading models, let's look at
some of their predictions.
```

Show hints Show solution Reset

Mark as completed Next

Part 1

- Process the text with the `nlp` object and create a `doc`.
- For each token, print the token text, the token's `.pos_` (part-of-speech tag) and the token's `.dep_` (dependency label).

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "It's official: Apple is the first U.S. public company to reach a $1 trillion mar

# Process the text
doc = nlp(text)

for token in doc:
    # Get the token text, part-of-speech tag and dependency label
    token_text = token.text
    token_pos = token.pos_
    token_dep = token.dep_
    # This is for formatting only
    print(f"{token_text:<12}{token_pos:<10}{token_dep:<10}")
```

Run code Submit

OUTPUT

```
It          PRON      nsubj
's          VERB      ccomp
official    NOUN      attr
:           PUNCT     punct
Apple       PROPN     nsubj
is          AUX       ROOT
the         DET       det
first       ADJ       amod
U.S.        PROPN     nmod
public      ADJ       amod
company     NOUN      attr
to          PART      aux
reach       VERB      relcl
a           DET       det
$           SYM       quantmod
1           NUM       compound
trillion    NUM       nummod
market      NOUN      compound
value       NOUN      dobj
✔ Perfect!
```

Show hints Show solution Reset

Part 2

- Process the text and create a `doc` object.
- Iterate over the `doc.ents` and print the entity text and `label_` attribute.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "It's official: Apple is the first U.S. public company to reach a $1 trillion mar

# Process the text
doc = nlp(text)

# Iterate over the predicted entities
for ent in doc.ents:
    # Print the entity text and its label
    print(ent.text, ent.label_)
```

Run code Submit

OUTPUT

```
Apple ORG
first ORDINAL
U.S. GPE
$1 trillion MONEY
```

Show hints Show solution Reset

Mark as completed Next

9 Predicting named entities in context

Models are statistical and not *always* right. Whether their predictions are correct depends on the training data and the text you're processing. Let's take a look at an example.

- Process the text with the `nlp` object.
- Iterate over the entities and print the entity text and label.
- Looks like the model didn't predict "iPhone X". Create a span for those tokens manually.

INPUT

```
import spacy

nlp = spacy.load("en_core_web_sm")

text = "Upcoming iPhone X release date leaked as Apple reveals pre-orders"

# Process the text
doc = nlp(text)

# Iterate over the entities
for ent in doc.ents:
    # Print the entity text and label
    print(ent.text, ent.label_)

# Get the span for "iPhone X"
iphone_x = doc[1:3]

# Print the span text
print("Missing entity:", iphone_x.text)
```

Run code | Submit

OUTPUT

```
Apple ORG
Missing entity: iPhone X
✔ Perfect! Of course, you don't always have to do this manually. In the
next exercise, you'll learn about spaCy's rule-based matcher, which can help you
find certain words and phrases in text.
```

Show hints | Show solution | Reset

Mark as completed | Next

11 Using the Matcher

Let's try spaCy's rule-based `Matcher`. You'll be using the example from the previous exercise and write a pattern that can match the phrase "iPhone X" in the text.

- Import the `Matcher` from `spacy.matcher`.
- Initialize it with the `nlp` object's shared `vocab`.
- Create a pattern that matches the `"TEXT"` values of two tokens: `"iPhone"` and `"X"`.
- Use the `matcher.add` method to add the pattern to the matcher.
- Call the matcher on the `doc` and store the result in the variable `matches`.
- Iterate over the matches and get the matched span from the `start` to the `end` index.

INPUT

```
import spacy

# Import the Matcher
from spacy.matcher import Matcher

nlp = spacy.load("en_core_web_sm")
doc = nlp("Upcoming iPhone X release date leaked as Apple reveals pre-orders")

# Initialize the Matcher with the shared vocabulary
matcher = Matcher(nlp.vocab)

# Create a pattern matching two tokens: "iPhone" and "X"
pattern = [{"TEXT": "iPhone"}, {"TEXT": "X"}]

# Add the pattern to the matcher
matcher.add("IPHONE_X_PATTERN", [pattern])

# Use the matcher on the doc
matches = matcher(doc)
print("Matches:", [doc[start:end].text for match_id, start, end in matches])
```

Run code | Submit

OUTPUT

```
Matches: ['iPhone X']
✔ Well done! You successfully found one match: the tokens at doc[1:3]
describing the span for "iPhone X".
```

Show hints | Show solution | Reset

Mark as completed | Next

12 Writing match patterns

In this exercise, you'll practice writing more complex match patterns using different token attributes and operators.

Part 1

- Write **one** pattern that only matches mentions of the *full* iOS versions: "iOS 7", "iOS 11" and "iOS 10".

INPUT

```
import spacy
from spacy.matcher import Matcher

nlp = spacy.load("en_core_web_sm")
matcher = Matcher(nlp.vocab)

doc = nlp(
    "After making the iOS update you won't notice a radical system-wide "
    "redesign: nothing like the aesthetic upheaval we got with iOS 7. Most of "
    "iOS 11's furniture remains the same as in iOS 10. But you will discover "
    "some tweaks once you delve a little deeper."
)

# Write a pattern for full iOS versions ("iOS 7", "iOS 11", "iOS 10")
pattern = [{"TEXT": "iOS"}, {"IS_DIGIT": True}]

# Add the pattern to the matcher and apply the matcher to the doc
matcher.add("IOS_VERSION_PATTERN", [pattern])
matches = matcher(doc)
print("Total matches found:", len(matches))

# Iterate over the matches and print the span text
for match_id, start, end in matches:
    print("Match found:", doc[start:end].text)
```

Run code | Submit

OUTPUT

```
Total matches found: 3
Match found: iOS 7
Match found: iOS 11
Match found: iOS 10
✔ Well done!
```

Show hints | Show solution | Reset

Part 2

- Write **one** pattern that only matches forms of "download" (tokens with the lemma "download"), followed by a token with the part-of-speech tag `"PROPN"` (proper noun).

INPUT

```
import spacy
from spacy.matcher import Matcher

nlp = spacy.load("en_core_web_sm")
matcher = Matcher(nlp.vocab)

doc = nlp(
    "i downloaded Fortnite on my laptop and can't open the game at all. Help? "
    "so when I was downloading Minecraft, I got the Windows version where it "
    "is the '.zip' folder and I used the default program to unpack it... do "
    "I also need to download Winzip?"
)

# Write a pattern that matches a form of "download" plus proper noun
pattern = [{"LEMMA": "download"}, {"POS": "PROPN"}]

# Add the pattern to the matcher and apply the matcher to the doc
matcher.add("DOWNLOAD_THINGS_PATTERN", [pattern])
matches = matcher(doc)
print("Total matches found:", len(matches))

# Iterate over the matches and print the span text
for match_id, start, end in matches:
    print("Match found:", doc[start:end].text)
```

Run code Submit

OUTPUT

```
Total matches found: 3
Match found: downloaded Fortnite
Match found: downloading Minecraft
Match found: download Winzip
✔ Good job!
```

Show hints Show solution Reset

Part 3

- Write **one** pattern that matches adjectives (`"ADJ"`) followed by one or two `"NOUN"`s (one noun and one optional noun).

INPUT

```
import spacy
from spacy.matcher import Matcher

nlp = spacy.load("en_core_web_sm")
matcher = Matcher(nlp.vocab)

doc = nlp(
    "Features of the app include a beautiful design, smart search, automatic "
    "labels and optional voice responses."
)

# Write a pattern for adjective plus one or two nouns
pattern = [{"POS": "ADJ"}, {"POS": "NOUN"}, {"POS": "NOUN", "OP": "?"}]

# Add the pattern to the matcher and apply the matcher to the doc
matcher.add("ADJ_NOUN_PATTERN", [pattern])
matches = matcher(doc)
print("Total matches found:", len(matches))

# Iterate over the matches and print the span text
for match_id, start, end in matches:
    print("Match found:", doc[start:end].text)
```

Run code | Submit

OUTPUT

```
Total matches found: 5
Match found: beautiful design
Match found: smart search
Match found: automatic labels
Match found: optional voice
Match found: optional voice responses
✔ Great work - those were some pretty complex patterns! Let's move on
to the next chapter and take a look at how to use spaCy for more advanced text
analysis.
```

Show hints | Show solution | Reset

Mark as completed | Next

Chapter 1: Finding words, phrases, names and concepts

This chapter will introduce you to the basics of text processing with spaCy. You'll learn about the data structures, how to work with trained pipelines, and how to use them to predict linguistic features in your text.

1. Introduction to spaCy 
2. Getting Started
3. Documents, spans and tokens
4. Lexical attributes
5. Trained pipelines 
6. Pipeline packages
7. Loading pipelines
8. Predicting linguistic annotations
9. Predicting named entities in context
10. Rule-based matching 
11. Using the Matcher
12. Writing match patterns